Pipelines Without Pipelines

How we built a data-pipeline engine with no pipeline object

A finite state machine you never wrote, implemented by the scripts themselves

First, the thing we're contrasting against

Windmill = run scripts (TS/Python/Go/SQL/...) as jobs.
Flows = wire scripts into a workflow.

A flow is a **declarative DAG**, fully defined upfront:

```
FlowValue {
  modules: [ FlowModule, FlowModule, ... ]   // the whole graph, as data
}
```

Branches, loops, retries, suspends, sub-flows — all just **data** in that tree.

`backend/windmill-types/src/flows.rs`

Flows are a finite state machine

Two structures, mirror images:

Definition (static)	State (dynamic)
`FlowValue.modules[]`	`FlowStatus.modules[]`
the graph	a cursor + per-step status
`flows.rs`	`flow_status.rs`

```
enum FlowStatusModule {
  WaitingForPriorSteps, WaitingForExecutor,
  InProgress, Success, Failure, WaitingForEvents,
}
```

`FlowStatus { step: i32, modules: [...] }` → **persisted as JSONB after every step.**

The question

The engine is **generic**. The graph lives in the **definition**.

> So what if there were **no definition**?

What if the graph were an **emergent property** of the scripts —
never written down anywhere?

Data pipelines: there is no pipeline

No `PipelineValue`. No DAG file. No orchestrator.

Just scripts that declare what they **touch**:

```
-- producer.sql                    -- consumer.sql
ATTACH 'datatable://main' AS db;   ATTACH 'datatable://main' AS db;
CREATE TABLE db.orders AS          -- on datatable://main/orders
  SELECT ... ;                     CREATE TABLE db.report AS
                                     SELECT ... FROM db.orders;
```

`producer` **writes** `orders`. `consumer` **reads** `orders`.
That edge is never declared. It's **parsed out of the SQL.**

The graph emerges from reads & writes

At deploy, we parse each script and write rows:

```
asset(path, kind, access_type, usage_path)
  orders   datatable   W   f/producer
  orders   datatable   R   f/consumer
```

The graph endpoint just `SELECT`**s and joins these back together.**
No compile step. Rename a script → graph updates instantly.

```
   [trigger] → producer —writes—▶ (orders) —reads—▶ consumer —▶ (report)
```

Execution: dispatch, not orchestration

A flow has a conductor stepping `i → i+1`.
A pipeline has **none.** It's a chain reaction:

```
// after ANY top-level script succeeds:
dispatch_asset_triggers(job)
  → for each asset the script WROTE
      → find scripts subscribed to it (// on <asset>)
        → push a job for each
```

`backend/windmill-queue/src/asset_dispatch.rs`

> Producer doesn't know its consumers. Consumers don't know each other.
> **Subscriptions alone define the graph.**

How is this even safe to do?

Both halves need to agree on "what does this script read/write":

- **Backend**, at deploy → must be **authoritative** (it drives real job dispatch)
- **Frontend**, as you type → must be **instant** (live graph in the editor)

Two implementations = two sources of truth = drift = bugs.

So we don't have two implementations.

We were already parsing every script

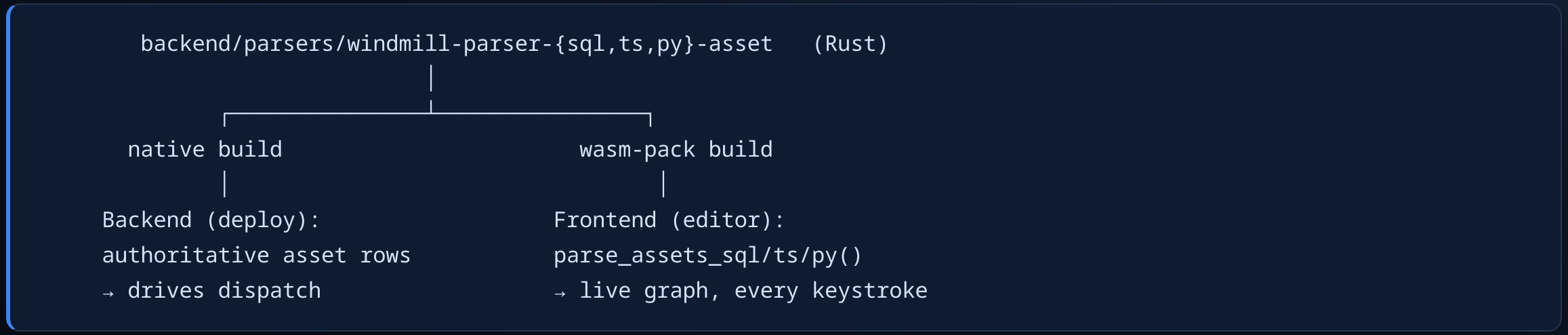
Every Windmill script is **already parsed** to derive its `main` signature — which powers the **input form** (params → JSON Schema → UI) and the **dependencies** (imports → lockfile). That parser already runs **native** (backend) + **WASM** (editor).

```
export async function main(
  name: string,
  count = 3,
  db: Postgresql,          // a Windmill resource type
) { /* ... */ }
```

↳ a JSON Schema → an auto-form: text field · number (default `3`) · resource picker.

> **The eureka:** assets are just *one more visitor* on the same AST — no new infrastructure, just a parser we'd already shipped, hardened, and dual-compiled for years.

One parser, compiled twice



Same parser, one more visitor. SQL via a real DuckDB-dialect parser; TS/Python via real ASTs (track `ATTACH ... AS db` / `wmill.datatable('main')`, then resolve usages).

That's the whole trick

The WASM parser is what makes "no pipeline object" *viable*:

- It can **implement** the graph — backend dispatch off parsed assets
- It can **represent** the graph — frontend draws it live from the same parse

The script definitions *are* the pipeline.
The parser is the bridge that lets one artifact be both **executable** and **drawable**.

Demo: watch the graph build itself

1. Open a pipeline folder → **graph reconstructed from a SQL query**
2. Type `CREATE TABLE clean AS SELECT … FROM orders`
   → WASM parses → **new node + edge appear, live**
3. Add `// on datatable://main/orders` → **subscription edge wires up**
4. **Deploy** → asset rows written → **run the source**
5. Watch the **cascade animate** down the graph (Activity feed groups the run)
6. Same DAG in the terminal: `wmill pipeline show f/orders`

The honest tradeoffs

Emergent graphs are great until they're spooky. So we made the implicit **visible**:

- **Cycle detection** — a write that loops back to its own producer is caught and broken, so cascades can't run away
- **AND-joins**: a script can wait for *all* inputs (partitions via `{partition}` token)
- `dispatch_event` **table** logs every decision: *dispatched / join-pending / skipped (why)* — surfaced on the job UI
- Backend re-parses at deploy → **server is authoritative**, browser can't lie
- Self-loops, debounce windows, retries — all declared in-script

Takeaways

Flows → explicit FSM. You write the graph; a generic interpreter walks it.
Pipelines → emergent graph. You write scripts; the graph *falls out* of them.

The unlock: **one parser, native + WASM**, so the script definitions are simultaneously what **runs** and what you **see**.

> 🚀 **We're hiring** — Rust / TypeScript / systems engineers → **windmill.dev/careers**

Code: `windmill-queue/src/asset_dispatch.rs` · `parsers/windmill-parser-*-asset` · `frontend/.../AssetGraph` — all on `main`.

Appendix A — The real code

The actual source behind each claim. Excerpts are verbatim, lightly trimmed where noted.

A1 · "Subscriptions alone define the graph"

Straight from the dispatch module's own doc comment — **verbatim**:

```
//! When a script writes an asset and a downstream script subscribes to
//! that asset via `// on s3://...`, this module pushes a job for each
//! subscriber after the producer's job completes successfully. Any
//! asset-writing top-level script cascades — there is no `// pipeline`
//! gate on the producer side; subscriptions alone define the graph.
```

`backend/windmill-queue/src/asset_dispatch.rs:11`

A2 · The dispatch loop (lightly trimmed)

```
let producers = workspace_producer_writes(db, &job.workspace_id).await?;   // cached
let Some(writes) = producers.get(runnable_path).cloned() else { return Ok(..) };

for (asset_kind, asset_path) in writes {
    let trigger_ref = format!("{}{}", prefix, asset_path);
    let subs = fetch_subscribers(db, &job.workspace_id, &trigger_ref).await?;
    for sub in subs {
        if sub.path == runnable_path { continue; }          // skip self-loops
        if sub.join_all { /* AND-join: wait for all inputs */ }
        push_subscriber(db, job, &sub.path, asset_kind, &asset_path,
                        runnable_path, depth + 1, /* partition, debounce, retry */).await?;
    }
}
```

`asset_dispatch.rs::try_dispatch` (273). Non-producers cost **one in-memory lookup, zero queries.**

A3 · One function, native AND wasm

The **WASM export** is a four-line shim — it forwards to the same crate the backend links natively:

```rust
#[cfg(feature = "asset-parser")]
#[wasm_bindgen]
pub fn parse_assets_sql(code: &str) -> String {
    match windmill_parser_sql_asset::parse_assets(code) {   // ← same fn the backend calls
        Ok(r)  => serde_json::to_string(&r).unwrap(),
        Err(e) => format!("err: {:?}", e),
    }
}
```

> `windmill-parser-wasm/src/lib.rs` — same pattern for `parse_assets_ts` / `parse_assets_py`.

A4 · ...and it's a real parser

`parse_assets` isn't a regex — it runs the **actual DuckDB-dialect parser** and walks the AST:

```rust
pub fn parse_assets(input: &str) -> anyhow::Result<ParseAssetsOutput> {
    let statements = Parser::parse_sql(&DuckDbDialect, input)?;
    let mut collector = AssetCollector::new();
    for statement in statements {
        let _ = statement.visit(&mut collector);
    }
    // → reads/writes resolved from ATTACH bindings + table refs
}
```

`backend/parsers/windmill-parser-sql-asset/src/asset_parser.rs`

A5 · The hard case: data tables in TypeScript

A TS script reaches a data table through the SDK — the parser reads the **TypeScript** *and* the **SQL embedded inside it**:

```ts
import * as wmill from "windmill-client"
export async function main(since: string) {
  const sql = wmill.datatable("main")        // bind: sql → datatable "main"
  await sql`
    SELECT * FROM orders WHERE created_at > ${since}
  `.fetch()                                  // READ   main/orders
  await sql`
    CREATE TABLE daily_revenue AS
    SELECT day, sum(total) FROM orders GROUP BY day
  `.fetch()                                  // WRITE  main/daily_revenue
}
```

Inferred: `datatable://main/orders` **R** · `datatable://main/daily_revenue` **W** — the TS AST resolves the `sql` binding, lifts each template's SQL, and runs it through the **same SQL parser**. *A parser inside a parser.*

A6 · The browser side calls it on every keystroke

```
import initAssetParser, {
  parse_assets_ts, parse_assets_py, parse_assets_sql
} from 'windmill-parser-wasm-asset'

export async function inferAssets(language, code) {
  if (language === 'duckdb')       { await initWasmAsset(); return wrap(parse_assets_sql(code)) }
  if (language === 'bun' || 'deno'){ await initWasmAsset(); return wrap(parse_assets_ts(code))  }
  if (language === 'python3')      { await initWasmAsset(); return wrap(parse_assets_py(code))  }
}
```

`frontend/src/lib/infer.ts` — wired into `ScriptEditor.svelte` as a reactive `resource([lang, code])`.

Appendix B — Drawing the emergent graph

The graph has no stored layout either — it's computed every render.

B1 · Band-reserving tidy-tree layout

The graph is **reconstructed from a SQL query**, so it also has to be **laid out from scratch** each time. Naive layered (Sugiyama) layout let unrelated edges fight for columns. The fix, **verbatim from the source**:

```
// Each node reserves a horizontal band at least as wide as the sum of its
// children's bands (W(n) = max(NODE_WIDTH, Σ W(children) + gaps)) ...
// Bands are exclusive, so two nodes with different parents can never
// interleave horizontally — the failure mode of the previous Sugiyama layout.
//
// The band recursion stops at *join points*: a node with two or more parents
// belongs to no single parent's band ... Joins instead root their own subtree.
```

`frontend/.../AssetGraph/assetGraphLayout.ts` · falls back to a grid on cyclic input.

B2 · Edges route around obstacles

A long edge that skips layers would draw **through** unrelated node boxes. So we detour:

- sample the straight line at each intervening row
- if a node sits under it (within `½·width + 8px`), it's a crossing
- route to the side the edge is already heading, outermost lane clears all crossings — **one detour, no box crossings**

`AssetGraphCanvas.svelte::detourForEdge` — computed **once per layout** (`$derived`), not per frame.

B3 · The live graph is a layered merge

What you see while editing = deployed graph **+** your unsaved reality, merged by precedence:

```
export function resolveGraph(input) {
  const acc = seedAccumulator(input, ctx)   // 1. base: /assets/graph (deployed)
  seedDraftOverlays(acc, input)             // 2. per-draft seeded triggers & outputs
  applyLiveBufferOverlay(acc, input, ctx)   // 3. the open script's live // on annotations
  crossCheckSweptScripts(acc, input)        // 4. reconcile renamed/removed
  overlayInferredLineage(acc, input)        // 5. WASM-inferred reads/writes (this keystroke)
}
```

`resolveGraph.ts` — base < session-inferred < draft < **live annotations** (highest wins).